

Đánh giá và phương pháp luận (Evaluation and Methodology)

PHỤ THUỘC A/05-observability-gauge-va-suy-bien (số bậc tự do gauge quyết định căn chỉnh mấy bậc — căn sai là che mất lỗi); A/06-bat-dinh-va-lan-truyen-sai-so (NEES và bệnh quá tự tin); B/11-loop-closure-va-nhat-quan-toan-cuc và B/12-relocalization (đánh giá có chi phí lỗi bất đối xứng).

TÓM TẮT

Mọi chương trước nợ chương này một câu trả lời: *làm sao biết một thay đổi thật sự là cải tiến?* Câu hỏi nghe đơn giản và nó bị trả lời sai một cách có hệ thống trong cả lĩnh vực lẫn trong thực hành công nghiệp. Nội dung có bốn phần. Một: các chỉ số quỹ đạo — ATE và RPE — với một chi tiết quyết định mà nhiều người làm sai: *căn chỉnh mấy bậc tự do*, và căn sai thì che mất đúng thứ ta muốn đo. Hai: đo *tính trung thực* chứ không chỉ độ chính xác, tức NEES — một phép đo rẻ mà gần như không ai làm. Ba: đánh giá các thành phần có chi phí lỗi bất đối xứng (loop closure, relocalization), nơi một con số trung bình là vô nghĩa và đường cong precision-recall ở vùng precision cao mới là thứ đáng nhìn. Bốn: kỷ luật phương pháp — ablation, mô phỏng, nhiều seed, và nguyên tắc quan trọng nhất của cả chương: *đánh giá ở đuôi phân bố, không ở trung bình*. Nếu chỉ nhớ một điều: *một hệ tốt hơn về trung bình nhưng tệ hơn ở đuôi là một hệ tệ hơn* — vì sản phẩm hỏng ở đuôi, không hỏng ở trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ATE** — *absolute trajectory error* — sai số vị trí giữa quỹ đạo ước lượng và chân trị sau khi căn chỉnh hai quỹ đạo. Đo sai số *toàn cục*, nên một loop closure tốt cải thiện nó nhiều.
- **RPE** — *relative pose error* — sai số của phép biến đổi tương đối trên các đoạn có độ dài cố định. Đo *drift cục bộ*, không phụ thuộc vào việc có đóng vòng hay không.
- **căn chỉnh (alignment)** — phép biến đổi áp lên quỹ đạo ước lượng để đưa nó về cùng hệ với chân trị, trước khi tính sai số. Số bậc tự do của phép biến đổi này *phải* bằng số bậc gauge của hệ.
- **ablation study** — thí nghiệm tắt từng thành phần một để đo đóng góp của nó. Chỉ hợp lệ khi mọi thứ khác được giữ nguyên và khi có đủ số lần chạy.
- **đuôi phân bố** — các trường hợp xấu nhất — phân vị 95, 99, hoặc trường hợp hỏng hoàn toàn. Đây là nơi hệ thống sản phẩm thất bại, và là nơi trung bình không nói gì.
- **ý nghĩa thống kê** — mức tin cậy rằng một khác biệt quan sát được không phải do ngẫu nhiên. SLAM có nguồn ngẫu nhiên thật (RANSAC, thứ tự luồng), nên một lần chạy không chứng minh gì.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đánh giá một hệ VIO bằng cách căn chỉnh $Sim(3)$ với chân trị rồi tính ATE — cách làm phổ biến. Vì sao đó là một sai lầm, và nó che mất điều gì cụ thể?
- Q2. Hệ thống mới của bạn có ATE trung bình thấp hơn hệ cũ 15% trên mười chuỗi. Nêu ba lý do vì sao điều đó vẫn chưa chứng minh nó tốt hơn.
- Q3. Đo NEES cần chân trị, nên nó chỉ làm được trong mô phỏng. Vì sao điều đó *không* phải lý do để bỏ qua nó, và nó cho biết gì mà ATE không cho biết?

- Q4. Bạn so hệ của mình với một baseline mà bạn tự cài đặt, và bạn thắng. Nêu phép kiểm bắt buộc trước khi tin kết quả đó.
- Q5. Hệ A có ATE trung bình 0,08 m và thất bại hoàn toàn trên 2% số lần chạy. Hệ B có ATE trung bình 0,12 m và chưa bao giờ thất bại. Hệ nào tốt hơn, và câu trả lời phụ thuộc vào gì?

1.1 Đặt vấn đề

Mười bảy chương trước đưa ra hàng chục lựa chọn thiết kế: detector nào, phần dư nào, kiến trúc back-end nào, chính sách bản đồ nào. Mỗi lựa chọn được biện minh bằng lập luận về cơ chế. Không lựa chọn nào được biện minh bằng số liệu — và đó là có chủ ý, vì chương này giải thích vì sao các số liệu phổ biến thường không biện minh được gì.

Bài toán gốc: *cho hai phiên bản của một hệ thống, làm sao biết cái nào tốt hơn?*

Nghe đơn giản. Nó khó vì bốn lý do chồng lên nhau.

Một — *“tốt hơn” không phải một chiều*. Một hệ có thể chính xác hơn nhưng chậm hơn, hoặc chính xác hơn ở trung bình nhưng hỏng nhiều hơn ở đuôi, hoặc chính xác hơn trong nhà và tệ hơn ngoài trời.

Hai — *chân trị khó có*. Hệ đo ngoài (motion capture) chỉ có trong phòng lab và chỉ trong một thể tích nhỏ. Ngoài đó, ta không biết sự thật.

Ba — *SLAM có tính ngẫu nhiên thật*. RANSAC lấy mẫu ngẫu nhiên; thứ tự luồng không tắt định; một khác biệt nhỏ trong thời điểm có thể đổi keyframe nào được chọn. Hai lần chạy cùng một hệ trên cùng dữ liệu cho hai kết quả khác nhau.

Bốn — *và đây là lý do nghiêm trọng nhất: điều ta đo thường không phải điều ta quan tâm*. Ta đo ATE trung bình trên các chuỗi mà hệ thống chạy được. Cái ta quan tâm là hệ thống có chạy được ở nhà khách hàng suốt tám tiếng hay không — và hai thứ đó tương quan yếu hơn nhiều so với vẻ ngoài.

Trước khi có các chỉ số chuẩn, mỗi bài báo tự định nghĩa cách đo, và so sánh giữa các bài là bất khả. TUM-RGBD [1] chuẩn hoá ATE và RPE; rpg_trajectory_evaluation [2] chuẩn hoá cách căn chỉnh và cách báo cáo drift theo đoạn. Đó là tiến bộ thật.

Nhưng chuẩn hoá chỉ số không giải quyết bốn lý do ở trên. Chương này lập luận rằng phần lớn giá trị của việc đánh giá nằm ở *thiết kế thí nghiệm*, không ở việc chọn chỉ số — và rằng thiết kế thí nghiệm là chỗ được đầu tư ít nhất.

TRỰC GIÁC

Hình dung bạn đánh giá hai tài xế.

Cách đơn giản: cho mỗi người lái một quãng đường và đo thời gian. Tài xế nhanh hơn thắng.

Ba chỗ sai của cách đó, và chúng ánh xạ chính xác sang ba chỗ sai của việc đánh giá SLAM.

Một — *bạn đo trên một quãng đường*. Tài xế A nhanh trên đường cao tốc, tài xế B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn định thuê người lái trong phố thì bạn vừa chọn sai.

Hai — *bạn đo trung bình, không đo tệ nhất*. Tài xế A nhanh hơn 10% nhưng thỉnh thoảng đâm xe. Tài xế B chậm hơn nhưng chưa bao giờ. Con số trung bình nói A thắng; bất kỳ ai thuê tài xế đều biết B mới là lựa chọn.

Ba — bạn không hỏi họ có biết mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và luôn nghĩ mình đang an toàn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe cho tới nay — nhưng họ không giống nhau chút nào.

Chuyện thứ ba là chuyện tình tế nhất và là chuyện mà gần như không ai đo. Nó tương ứng với việc đo *tính trung thực của hiệp phương sai*: một hệ biết mình sai bao nhiêu thì dùng được, một hệ luôn nghĩ mình đúng thì không — dù hai hệ có cùng ATE.

Và một chuyện thứ tư, về sự công bằng. Nếu bạn để tài xế A lái chiếc xe quen của anh ta và bắt tài xế B lái một chiếc xe lạ, thì kết quả không nói gì về hai tài xế. Trong đánh giá SLAM, chuyện này xảy ra liên tục dưới dạng: người đề xuất phương pháp mới tự cài đặt baseline.

1.2 Chỉ số quỹ đạo, và cái bẫy căn chỉnh

A. ATE và RPE đo hai thứ khác nhau

ATE căn chỉnh toàn bộ quỹ đạo ước lượng với chân trị rồi tính sai số vị trí ở mỗi thời điểm. Nó đo *sai số toàn cục*, nên nó rất nhạy với việc có đóng vòng hay không: một loop closure tốt cải thiện ATE mạnh.

RPE lấy các đoạn có độ dài cố định — chẳng hạn mọi đoạn 10 m — và đo sai số của phép biến đổi tương đối trên mỗi đoạn. Nó đo *drift cục bộ*, và nó gần như không bị ảnh hưởng bởi loop closure.

Hai chỉ số trả lời hai câu khác nhau, và dùng nhầm là một lỗi thật. Nếu bạn đang cải thiện odometry, RPE là chỉ số đúng — ATE sẽ bị chi phối bởi việc có đóng vòng hay không, và nó che mất đóng góp của bạn. Nếu bạn đang cải thiện loop closure, ATE là chỉ số đúng — RPE gần như không đổi.

Cách báo cáo tốt nhất, theo [2]: *drift theo đoạn*, tức RPE tính trên nhiều độ dài đoạn khác nhau và biểu diễn dưới dạng phần trăm quãng đường. Nó cho một bức tranh đầy đủ hơn một con số, và nó so sánh được giữa các chuỗi có độ dài khác nhau — điều mà ATE tuyệt đối không làm được.

B. Căn chỉnh mấy bậc tự do: cái bẫy chính

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là lỗi mà tôi cho là phổ biến nhất trong đánh giá SLAM.

Quỹ đạo ước lượng sống trong một hệ quy chiếu tùy ý — đó là gauge ở A/05. Trước khi so với chân trị, phải căn chỉnh. Câu hỏi: căn chỉnh *mấy bậc tự do*?

Câu trả lời: *đúng bằng số bậc gauge của hệ*, và A/05 mục 3B cho bảng:

Mono thuần thị giác: $Sim(3)$, bảy bậc — vì tỉ lệ là gauge tự do. Stereo: $SE(3)$, sáu bậc. VIO: *bốn* bậc — ba tịnh tiến và yaw, vì trọng lực ghim roll với pitch.

Căn sai thì sao? Hai hướng sai với hai hậu quả khác nhau.

Căn quá nhiều bậc — ví dụ căn $Sim(3)$ cho một hệ VIO — *che mất lỗi*. Phép căn chỉnh sẽ tự động co giãn quỹ đạo ước lượng cho khớp với chân trị, nên một hệ có sai số tỉ lệ 10% sẽ cho ATE giống hết một hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ bạn cần đo — và với VIO, tỉ lệ là một trong những thứ quan trọng nhất cần kiểm, vì nó là thứ IMU đóng góp.

Căn quá ít bậc — ví dụ căn bốn bậc cho một hệ mono — cho ATE lớn giả tạo, vì phần gauge chưa được hấp thụ sẽ xuất hiện thành sai số.

Hệ quả thực hành: *luôn ghi rõ căn chỉnh mấy bậc khi báo cáo ATE*, và nghi ngờ mọi bảng so sánh không ghi. Và nếu bạn đang so hai hệ có cấu hình cảm biến khác nhau — mono với VIO chẳng hạn — thì ATE của chúng *không so sánh trực tiếp được*, vì chúng phải căn chỉnh với số bậc khác nhau.

CẠM BÃY

Căn chỉnh $Sim(3)$ cho một hệ có IMU là cách phổ biến nhất để tự lừa mình.

Cơ chế: phép căn chỉnh cơ giãn quỹ đạo ước lượng cho khớp chân trị, nên nó *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — một lỗi nghiêm trọng, thường do khởi tạo kém hoặc do kích thích không đủ (B/10) — sẽ cho ATE gần như giống hệt một hệ có tỉ lệ đúng.

Bạn sẽ thấy một con số đẹp và kết luận hệ thống tốt, trong khi thứ mà IMU được thêm vào để cung cấp — đơn vị mét — đang sai.

Ba quy tắc để tránh. *Một*: căn chỉnh đúng số bậc theo cấu hình cảm biến, và **ghi rõ trong mọi báo cáo**. *Hai*: với hệ có tỉ lệ quan sát được, hãy **báo cáo riêng sai số tỉ lệ** như một con số độc lập — nó là thứ đáng theo dõi nhất và nó biến mất khỏi ATE nếu căn sai. *Ba*: khi đọc một bảng so sánh không ghi số bậc căn chỉnh, hãy coi các con số là không diễn giải được, chứ không phải là “chắc họ làm đúng”.

1.3 Đo tính trung thực, không chỉ độ chính xác

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là mục mà tôi cho là có tỉ lệ giá trị trên mức phổ biến cao nhất trong chương.

A. NEES và điều nó cho biết

Từ A/06 mục 4A: $\varepsilon = \tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}$, và nếu Σ trung thực thì $\mathbb{E}[\varepsilon]$ bằng số chiều.

ATE trả lời: *bạn sai bao nhiêu*. NEES trả lời: *bạn có thành thật về việc mình sai bao nhiêu không*. Hai câu hỏi khác nhau, và một hệ có thể tốt ở câu một và tệ ở câu hai — thực tế đó là trường hợp thường gặp (A/06 mục 4).

Vì sao câu hai quan trọng với sản phẩm: mọi module phía trên dùng hiệp phương sai để ra quyết định. Bộ lập kế hoạch quyết định có đi qua khe hẹp không; loop closure quyết định có chấp nhận một ứng viên không (B/11 mục 3B); relocalization quyết định có tin kết quả không (B/12 mục 4B). Ba quyết định ấy đều sai nếu hiệp phương sai quá nhỏ.

Cái giá: cần chân trị, nên chỉ làm được trong mô phỏng hoặc trên bộ dữ liệu có hệ đo ngoài.

B. Vì sao “cần chân trị” không phải lý do bỏ qua

Lập luận thường gặp: NEES chỉ đo được trong mô phỏng, mà mô phỏng không giống thực tế, nên nó không có giá trị.

Lập luận ấy sai ở một chỗ quan trọng. NEES trong mô phỏng không đo “hệ thống của tôi tốt tới đâu trong thực tế”. Nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình mà tôi đã giả định*. Đó là một câu hỏi về *tính đúng đắn của cài đặt*, và mô phỏng là chỗ lý tưởng để trả lời nó, vì ở đó mọi giả định đều thoả chính xác.

Nói cách khác: nếu NEES lệch trong mô phỏng — nơi nhiều đúng là Gauss, trọng số đúng là σ bạn đã dùng để sinh dữ liệu — thì bạn có *lỗi cài đặt*, không phải vấn đề mô hình. Nhầm hiệp phương sai biên với điều kiện (A/06 mục 2B), quên nhân Adjoint khi đổi quy ước (A/03), sai một

hệ số ở đầu đó. Đây là những lỗi mà không phép đo nào khác bắt được, và chúng *im lặng* — hệ thống vẫn cho ATE tốt.

Đây là lập luận quan trọng nhất của mục này: *mô phỏng không dùng để chứng minh hệ thống tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*. Chạy trong mô phỏng trước là cách rẻ nhất để biết phần nào của vấn đề là do bạn viết sai và phần nào là do thế giới khó.

C. Khi không có chân trị

Tại hiện trường, bốn đại lượng thay thế đáng theo dõi, và chúng là nền cho giám sát sức khoẻ ở D/21: số inlier và tỉ lệ inlier; số điều kiện hoặc giá trị riêng nhỏ nhất của Λ ; vết của hiệp phương sai tư thế; và tỉ lệ tracking.

Không đại lượng nào trong bốn cái đó đo sai số. Chúng đo các *điều kiện* mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng và sai số thật phải được *hiệu chuẩn một lần* trên dữ liệu có chân trị, rồi dùng ngoài hiện trường. Đó là một quy trình đáng làm và ít ai làm.

1.4 Đánh giá thành phần có chi phí lỗi bất đối xứng

Loop closure (B/11) và relocalization (B/12) có chung một đặc điểm: chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc. Đánh giá chúng bằng một con số duy nhất là vô nghĩa.

A. Precision-recall, và vùng đáng nhìn

Cách đúng là đường cong precision-recall. Nhưng — và đây là điểm quan trọng — *điểm F1 tối đa là tiêu chí sai*, vì F1 giả định precision và recall có giá trị ngang nhau.

Với chi phí bất đối xứng, vùng đáng nhìn là vùng *precision rất cao*. Câu hỏi đúng là: ở *precision 99,9%*, *recall của tôi là bao nhiêu*? Một hệ có recall 60% ở precision 99,9% hữu ích hơn một hệ có recall 95% ở precision 95%, dù con số thứ hai trông đẹp hơn — vì 5% dương tính giả nghĩa là bản đồ bị phá định kỳ.

Cách báo cáo tốt: chọn một mức precision từ *yêu cầu sản phẩm* — “tôi chấp nhận một loop closure sai trên một nghìn” — rồi báo cáo recall ở mức đó. Con số ấy có nghĩa và so sánh được.

B. Đánh giá trên truy vấn không có câu trả lời

Một thiếu sót có hệ thống của các benchmark, đã nêu ở B/12 khối *Cạm bẫy*: chúng chỉ chứa các truy vấn *có* câu trả lời trong bản đồ.

Trong thực tế, phần lớn truy vấn relocalization xảy ra ở những chỗ mà hệ thống *chưa* lập bản đồ — robot bị đưa sang một phòng mới, hoặc bản đồ đã lỗi thời. Hành vi đúng là trả về “không biết”. Không benchmark nào đo điều đó.

Cách khắc phục cho bộ dữ liệu nội bộ: cố ý đưa vào các truy vấn ngoài bản đồ và đo tỉ lệ hệ thống trả lời sai thay vì nói không biết. Đây là một trong những phép đo có giá trị cao nhất cho sản phẩm và gần như không tồn tại trong tài liệu.

1.5 Kỷ luật phương pháp

A. Nhiều lần chạy và ý nghĩa thống kê

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

SLAM có nguồn ngẫu nhiên thật: RANSAC lấy mẫu ngẫu nhiên, thứ tự luồng không tắt định, và một khác biệt nhỏ về thời điểm đổi keyframe nào được chọn. Độ tán giữa các lần chạy có thể so sánh được với khác biệt giữa hai phương pháp.

Quy tắc tối thiểu: *chạy mỗi cấu hình nhiều lần với các seed khác nhau, và báo cáo cả trung vị lẫn độ tán. Một cải thiện 15% trong khi độ tán giữa các lần chạy là 20% thì chưa chứng minh gì.*

Ba lý do vì sao 15% trên mười chuỗi vẫn chưa đủ: *một*, có thể nằm trong độ tán chạy-tới-chạy; *hai*, có thể do một hoặc hai chuỗi chi phối trong khi các chuỗi khác tệ đi — nên phải nhìn phân bố chứ không nhìn trung bình; *ba*, mười chuỗi có thể không đại diện cho môi trường triển khai.

Một cách rẻ để loại nguồn ngẫu nhiên: làm hệ thống *tắt định* — seed cố định, thứ tự xử lý cố định. Điều này đáng làm vì lý do khác quan trọng hơn: nó là điều kiện để debug một lỗi từ hiện trường (D/20).

B. Baseline phải tử tế

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là một quy tắc mà `research-rules.md` mục 7 nêu rõ.

Nếu bạn tự cài đặt baseline và bạn thắng, phép kiểm bắt buộc là: *baseline của tôi có đạt được con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu, và kết quả không nói gì.

Đây không phải một cảnh báo lý thuyết. Rất nhiều cải tiến biến mất khi baseline được chỉnh tử tế, và lý do có cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline.

Quy tắc thực hành: dùng mã nguồn gốc của baseline khi có; nếu phải tự cài, hãy tái lập được con số công bố *trước* khi so sánh; và nếu không tái lập được, hãy ghi rõ điều đó thay vì im lặng.

C. Ablation có kỷ luật

Tắt từng thành phần một, giữ nguyên mọi thứ khác, và đo. Nghe hiển nhiên, và nó hay bị làm sai theo ba cách.

Sai một — *tắt nhiều thứ cùng lúc*. Nếu tắt cả loop closure lẫn local BA, bạn không biết cái nào đóng góp gì.

Sai hai — *không chỉnh lại tham số*. Tắt một thành phần có thể làm các tham số tối ưu thay đổi. Nếu bạn tắt M-estimator mà giữ nguyên ngưỡng RANSAC, bạn đang đo một cấu hình chưa được tinh chỉnh, không phải đóng góp của M-estimator.

Sai ba — *không chạy đủ nhiều lần*. Áp dụng đúng như mục 5A.

D. Đánh giá ở đuôi, không ở trung bình

Đây là nguyên tắc quan trọng nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ năm.

Với hệ A (ATE trung bình 0,08 m, thất bại 2%) so với hệ B (ATE 0,12 m, chưa bao giờ thất bại): câu trả lời phụ thuộc vào *chi phí của một lần thất bại*.

Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên hệ A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật. Hệ B thắng, và thắng không sát sao.

Đây là chỗ mục tiêu sản phẩm ở 00-map tạo ra một khác biệt thật về phương pháp: *SOTA thật nằm ở đuôi phân bố, không ở trung bình*. Ba hệ quả cụ thể.

Một — báo cáo phân vị, không chỉ trung bình. Phân vị 95 và 99 của ATE, và của thời gian mỗi khung. B/09 mini project đã nhấn mạnh điều này cho thời gian.

Hai — đếm và phân loại thất bại. Số lần mất bám, số lần relocalization sai, số lần bản đồ bị phá. Các con số này thường không xuất hiện trong bài báo và chúng là con số quan trọng nhất cho sản phẩm.

Ba — thiết kế thí nghiệm nhắm vào đuôi. Đừng chỉ chạy trên các chuỗi mà hệ thống chạy được. Chạy trên các chuỗi khó, các chuỗi có sự cố, các chuỗi ngoài phân bố. D/21 lập luận rằng đây là lý do phải có bộ dữ liệu nội bộ phản ánh môi trường khách hàng.

CẦN NHỚ

Bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và ba câu sau thường bị bỏ.

Một — chính xác tới đâu? ATE và RPE, căn chỉnh đúng số bậc. Đây là câu duy nhất mà phần lớn bài báo trả lời.

Hai — có thành thật không? NEES trong mô phỏng để kiểm cài đặt hiệp phương sai; các đại lượng thay thế ngoài hiện trường.

Ba — hỏng thế nào và bao nhiêu lần? Tỷ lệ thất bại, phân loại nguyên nhân, và hành vi khi hỏng — nó dừng lại hay tiếp tục báo số sai?

Bốn — ở đuôi thì sao? Phân vị 95 và 99, không phải trung bình. Với sản phẩm, đây là câu quan trọng nhất.

Và một nguyên tắc bao trùm: **đo trước, cải tiến sau**. Cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ — bạn sẽ dành ba tháng làm giảm ATE trung bình 10% trong khi vấn đề thật là 2% số lần chạy bị hỏng vì một nguyên nhân bạn chưa từng đo.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
evo [3]	Công cụ tính ATE, RPE với các chế độ căn chỉnh khác nhau	Dễ dùng; chú ý chọn đúng chế độ căn chỉnh — mặc định không phải lúc nào cũng đúng cho hệ của bạn
rpg_trajectory_evaluation [2]	Căn chỉnh theo cấu hình cảm biến; drift theo đoạn; nhiều lần chạy	Cài đặt tham chiếu của mục 2; đọc tài liệu về lựa chọn căn chỉnh
OpenVINS [4]	Mô phỏng có chân trị; tính NEES trực tiếp	Một trong ít hệ mã nguồn mở đo tính nhất quán chứ không chỉ độ chính xác
TUM-RGBD [1]	Định nghĩa chuẩn của ATE và RPE	Nguồn của mục 2A
OpenLORIS [5]	Bộ dữ liệu nhiều phiên, môi trường thay đổi	Một trong ít bộ dữ liệu cho phép đánh giá hoạt động dài hạn

MINI PROJECT SLAM · Đánh giá lại một kết quả bạn đã tin, theo bốn câu hỏi ở khối Cần nhớ · 3 ngày

Mục tiêu	phát hiện xem một kết luận bạn đã rút ra từ các chương trước có đứng vững dưới một đánh giá có kỷ luật hay không.
Dữ liệu	chọn một mini project bạn đã làm ở các chương trước — so ba phương pháp triangulation, hay so hai front-end, hay so ba back-end — và dùng lại dữ liệu đó, cộng thêm vài chuỗi khó.

Các bước	chạy lại so sánh cũ nhưng lần này: (a) căn chỉnh đúng số bậc theo cấu hình cảm biến và ghi rõ; (b) chạy mỗi cấu hình <i>mười</i> lần với seed khác nhau và báo cáo trung vị cùng khoảng phân vị; (c) báo cáo phân vị 95 và 99 chứ không chỉ trung bình; (d) đếm và phân loại các lần thất bại; (e) nếu có mô phỏng, tính NEES.
Xong khi	bạn trả lời được: kết luận cũ của bạn có còn đúng không, và nếu có thì với mức tin cậy nào? Kết quả có giá trị nhất của bài này là khi câu trả lời là <i>không</i> — bạn vừa tránh được việc xây tiếp trên một kết luận sai. Nếu độ tán giữa mười lần chạy lớn hơn khác biệt giữa hai phương pháp, hãy ghi điều đó vào 99-chua-biet.md như một câu hỏi mở thay vì kết luận bừa.
Nối với	D/21-tu-prototype-len-mass-production biến quy trình này thành regression test tự động chạy mỗi commit; D/22-huong-nghien-cuu-con-trong dựa vào nó để phân biệt một khoảng trống thật với một khoảng trống tưởng tượng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì *Sim(3)* có bảy bậc gồm cả *tỉ lệ*, nên phép căn chỉnh sẽ co giãn quỹ đạo ước lượng cho khớp chân trị và *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — lỗi nghiêm trọng, thường do khởi tạo kém hoặc kích thích không đủ — sẽ cho ATE gần giống hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ cần đo, và đó là thứ mà IMU được thêm vào để cung cấp. Với VIO phải căn *bốn* bậc (ba tịnh tiến + yaw), vì trọng lực ghim roll và pitch. Và nên báo cáo sai số tỉ lệ như một con số riêng.
- A2. (1) Có thể nằm trong *độ tán chạy-tới-chạy* — SLAM có ngẫu nhiên thật từ RANSAC và thứ tự luồng, và độ tán có thể so sánh được với 15%. (2) Có thể do *một hoặc hai chuỗi chi phối* trong khi các chuỗi khác tẻ đi — phải nhìn phân bố chứ không nhìn trung bình. (3) Mười chuỗi có thể *không đại diện* cho môi trường triển khai. Và một lý do thứ tư quan trọng hơn cả ba: 15% ở *trung bình* không nói gì về *đuôi*, và sản phẩm hỏng ở đuôi. Cách kiểm: chạy nhiều seed, báo cáo trung vị và độ tán, báo cáo phân vị 95 và 99, và đếm số lần thất bại.
- A3. Vì NEES trong mô phỏng không đo “hệ tôi tốt tới đâu trong thực tế” — nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình tôi đã giả định*. Mô phỏng là *chỗ lý tưởng* cho câu hỏi ấy vì ở đó mọi giả định thoả chính xác. Nếu NEES lệch trong mô phỏng thì bạn có *lỗi cài đặt* — nhằm hiệp phương sai biên với điều kiện, quên Adjoint khi đổi quy ước, sai một hệ số — và đó là loại lỗi im lặng mà không phép đo nào khác bắt được, vì ATE vẫn tốt. Nguyên tắc: *mô phỏng không dùng để chứng minh hệ tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*.
- A4. Phép kiểm bắt buộc: *baseline của tôi có đạt con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu và kết quả không nói gì. Đây không phải cảnh báo lý thuyết mà có lý do cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline. Quy tắc: dùng mã nguồn gốc khi có; nếu tự cài thì phải tái lập được con số công bố *trước* khi so sánh; nếu không tái lập được thì ghi rõ điều đó thay vì im lặng.
- A5. Phụ thuộc vào *chi phí của một lần thất bại*. Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật, chưa kể thiệt hại về uy tín. B thắng, và không sát sao. Với mục tiêu sản phẩm ở 00-map, nguyên tắc là *SOTA thật nằm ở đuôi phân bố*: báo cáo phân vị thay vì trung bình, đếm và phân loại thất bại, và thiết kế thí nghiệm nhắm vào các chuỗi khó chứ không chỉ các chuỗi chạy được.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn thuê một tài xế và có hai ứng viên. Cách đơn giản nhất: cho mỗi người lái một quãng, đo thời gian, chọn người nhanh hơn.

Cách đó sai ở ba chỗ, và ba chỗ ấy chính là ba lỗi phổ biến nhất khi đánh giá một hệ định vị.

Thứ nhất, bạn đo trên một quãng đường. Người A nhanh trên cao tốc, người B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn cần người lái trong phố thì bạn vừa chọn nhầm.

Thứ hai, bạn đo trung bình chứ không đo lúc tệ nhất. Người A nhanh hơn mười phần trăm nhưng thỉnh thoảng đâm xe. Người B chậm hơn nhưng chưa bao giờ. Con số trung bình bảo chọn A; bất kỳ ai thật sự đi thuê tài xế đều biết phải chọn B.

Thứ ba, và tinh tế nhất: bạn không hỏi họ có *biết* lúc nào mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và lúc nào cũng nghĩ mình đang ổn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe *cho tới nay* — nhưng họ không giống nhau chút nào, và bạn sẽ biết điều đó vào một ngày không đẹp trời.

Cái thứ ba là cái gần như không ai đo. Nó tương ứng với việc hỏi một cái máy: “mày nói mày sai hai centimet — điều đó có thật không?” Rất ít người hỏi câu đó, và câu trả lời thường là không.

Còn một chuyện nữa, về sự công bằng khi so sánh. Nếu bạn để người A lái chiếc xe quen của anh ta và bắt người B lái một chiếc xe lạ, thì kết quả chẳng nói gì về hai người. Chuyện này xảy ra liên tục trong nghiên cứu, dưới dạng: người đề xuất phương pháp mới tự cài lại phương pháp cũ để so sánh — và dành hàng tháng cho cái của mình, vài ngày cho cái của người khác.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *không có sự thật để so* ngoài phòng thí nghiệm. Trong phòng lab có hệ đo ngoài cho biết chân trị; ở nhà khách hàng thì không. Nên mọi thứ ta đo được ngoài hiện trường đều là dấu hiệu gián tiếp, và việc nối các dấu hiệu ấy với sai số thật là một việc phải làm một lần trong lab rồi mang ra dùng — và rất ít nơi làm.

Một câu: *Đo trước rồi hãy cải tiến — và đo ở chỗ tệ nhất, không ở chỗ trung bình.*

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *cần chỉnh phải đúng số bậc gauge* — suy từ A/05 (cửa a) và được [2] thiết lập (cửa c). Nó kiểm được trực tiếp: lấy một quỹ đạo, nhân tỉ lệ 1,1, rồi tính ATE với căn $Sim(3)$ và với căn $SE(3)$; nếu con số thứ nhất không đổi thì mệnh đề được xác nhận.

Mệnh đề thứ hai — *NEES trong mô phỏng có giá trị vì nó tách lỗi cài đặt khỏi lỗi mô hình* — là một lập luận về mục đích của phép đo, không phải một sự kiện thực nghiệm. Nó bị bác bỏ nếu trong thực tế, các lỗi cài đặt hiệp phương sai hiểm tới mức phép đo này không đáng công. Tôi cho là ngược lại — tôi ngờ chúng rất phổ biến, dựa trên việc phân biệt biên với điều kiện là một điểm tinh tế — nhưng tôi không có số liệu (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ ba — *SOTA thật nằm ở đuôi* — phụ thuộc vào mục tiêu. Nó đúng cho sản phẩm robot hoạt động không giám sát, là mục tiêu đã nêu ở 00-map. Nó *sai* cho một ứng dụng mà thất bại rẻ và độ chính xác trung bình là thứ được trả tiền — quét đo đặc có người vận hành chẳng hạn, nơi một lần hỏng chỉ tốn một lần quét lại. Nếu rõ điều này vì nguyên tắc ở mục 5D không phổ quát.

Mệnh đề thứ tư — *benchmark không đo truy vấn ngoài bản đồ* — là một quan sát về các bộ dữ liệu hiện có, và nó bị bác bỏ nếu một benchmark mới đưa vào phần đó. Nếu điều đó xảy ra thì đó là tin tốt, và tôi mong nó xảy ra.

1.7 Kết nối

Chương này là chương mà mọi chương trước đều nợ, nên nổi lên trên là nổi tới tất cả.

Cụ thể: A/05-observability-gauge-va-suy-bien cho quy tắc căn chỉnh ở mục 2B — không có chương đó thì quy tắc này trông tùy tiện. A/06-bat-dinh-va-lan-truyen-sai-so cho NEES và cho lý do vì sao tính trung thực quan trọng. B/11-loop-closure-va-nhat-quan-toan-cuc và B/12-relocalization cho hai thành phần có chi phí lỗi bất đối xứng ở mục 4. C/15-hop-nhat-da-cam-bien khối *Cần nhớ* nêu rằng đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó — mục 5D ở đây là cách khắc phục.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ chương này trở thành quy trình: bộ dữ liệu nội bộ phản ánh môi trường khách hàng, regression test tự động cho mỗi commit, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. D/22-huong-nghien-cuu-con-trong phụ thuộc vào chương này ở một mức sâu hơn: không thể phân biệt một khoảng trống *thật* với một khoảng trống *tưởng tượng* nếu không có kỷ luật đo lường — và ba trong tám hướng ở đó (bất định đáng tin, dự đoán thất bại, xử lý suy biến có nguyên tắc) chỉ có nghĩa khi ta đo được chúng.

Nối sang research-rules.md: mục 5B và 5C của chương này là mục 7 của tài liệu đó áp dụng cho SLAM. Nếu bạn định nâng một phần của cây này lên thành một dự án khảo sát hoặc nghiên cứu, chương này là cầu nối.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Căn chỉnh mấy bậc cho mono, stereo, VIO — và nói căn quá nhiều bậc che mất điều gì cụ thể.
2. ATE và RPE đo hai thứ khác nhau. Nếu bạn đang cải thiện odometry thì dùng cái nào, và vì sao cái kia sẽ che mất đóng góp của bạn?
3. NEES cần chân trị nên chỉ đo được trong mô phỏng. Vì sao điều đó không phải lý do bỏ qua, và nó bắt được loại lỗi nào mà ATE không bắt được?
4. Vì sao điểm F1 tối đa là tiêu chí sai cho loop closure, và tiêu chí đúng là gì?
5. Nêu ba cách làm sai một ablation study.
6. Hệ A nhanh và thỉnh thoảng hỏng, hệ B chậm và chưa bao giờ hỏng. Câu trả lời phụ thuộc vào gì, và với mục tiêu của cây này thì chọn cái nào?
7. Nêu bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và nói vì sao ba câu sau thường bị bỏ.

Nguồn

- [1] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [2] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [3] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [4] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [5] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.